

LAPORAN PRAKTIKUM 5 STRUKTUR DATA
“SINGLE LINKED LIST”

Disusun Oleh:

Dinda Amelia

(2511531020)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG 2026

KATA PENGANTAR

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW, yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Laporan pratikum ini disusun sebagai bentuk penanggung jawaban atas pelaksanaan kegiatan pratikum mata kuliah struktur data yang membahas tentang Single Linked List di Java Eclipse. Dalam praktikum ini, penulis tidak hanya belajar tentang Penyusunan laporan ini bertujuan untuk memahami konsep dasar struktur data linked list.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I	1
PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
1.3 Manfaat Praktikum	1
BAB II.....	2
PEMBAHASAN	2
2.1 Dasar Teori	2
2.2 Langkah kerja	2
2.2.1 Buka eclipse.....	2
2.2.2 Membuat Java Project.....	3
2.2.3 Class 1 “NodeSLL_2511531020”.....	4
2.2.4 Class 2 “TambahSLL_2511531020”	7
2.2.5 Class 3 “PencarianSLL_2511531020”	11
2.2.6 Class 4 “HapusSLL_2511531020”	14
2.3 Tugas Praktikum.....	18
2.3.1 Class 1 “Pasien_2511531020”.....	18
2.3.2 Class 2 “RumahSakit_2511531020”.....	20
BAB III	25
KESIMPULAN	25
DAFTAR PUSTAKA	26

BAB I

PENDAHULUAN

1.1 Latar Belakang

Single Linked List (SLL) adalah salah satu struktur data dinamis yang digunakan untuk menyimpan sekumpulan elemen secara berurutan. Berbeda dengan array yang memiliki ukuran tetap, linked list memungkinkan penambahan dan penghapusan elemen secara fleksibel tanpa perlu menggeser elemen lain. Setiap elemen dalam linked list disebut node, yang terdiri dari dua bagian: data dan pointer (referensi) ke node berikutnya. Konsep ini membuat linked list menjadi solusi efisien untuk operasi yang sering melibatkan perubahan ukuran data.

1.2 Tujuan

1. Memahami konsep dasar struktur data linked list.
2. Mempelajari cara kerja node dalam menyimpan data dan pointer.
3. Mengimplementasikan operasi dasar seperti insert, delete, dan traversal pada linked list.
4. Menerapkan prinsip dinamis dalam pengelolaan memori sehingga lebih efisien dibanding array statis.

1.3 Manfaat Praktikum

1. Memberikan pemahaman mendalam tentang bagaimana data dapat disimpan dan dihubungkan secara dinamis, sehingga mahasiswa lebih siap menghadapi struktur data kompleks.
2. Melatih logika pemrograman dalam mengelola pointer/referensi, yang berguna untuk implementasi algoritma berbasis hubungan antar data.
3. Memberikan keterampilan praktis untuk menambah, menghapus, dan menelusuri data secara efisien tanpa bergantung pada array statis.
4. Menghasilkan program yang lebih fleksibel dan efisien dalam penggunaan memori, karena ukuran linked list dapat berubah sesuai kebutuhan.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Node: elemen dasar linked list yang terdiri dari data dan next.

Head: pointer yang menunjuk ke node pertama dalam list.

Tail: node terakhir yang menunjuk ke null.

Operasi dasar:

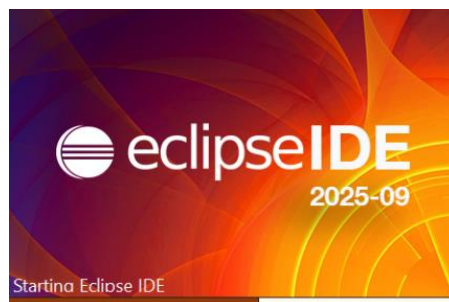
- Insert: menambahkan node di depan, belakang, atau posisi tertentu.
- Delete: menghapus node dari list.
- Traversal: menelusuri seluruh node dari head hingga tail.

Karakteristik utama:

- Bersifat dinamis (tidak memiliki ukuran tetap).
- Menggunakan pointer untuk menghubungkan antar node.
- Mengikuti struktur linear, tetapi tidak berbasis indeks seperti array.

2.2 Langkah kerja

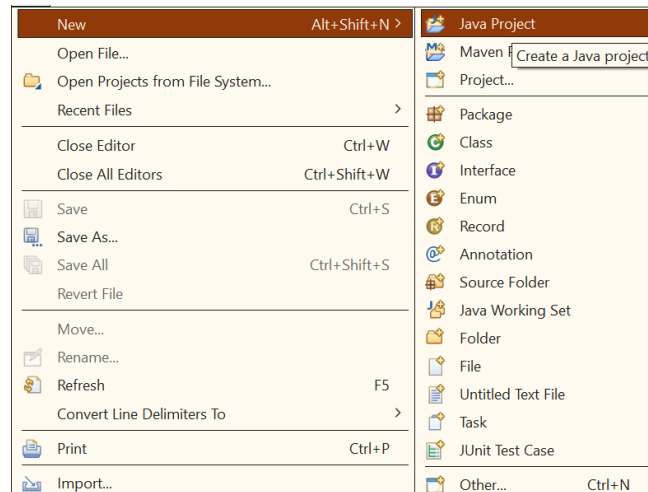
2.2.1 Buka eclipse



Gambar 2.1 Buka eclipse jalankan

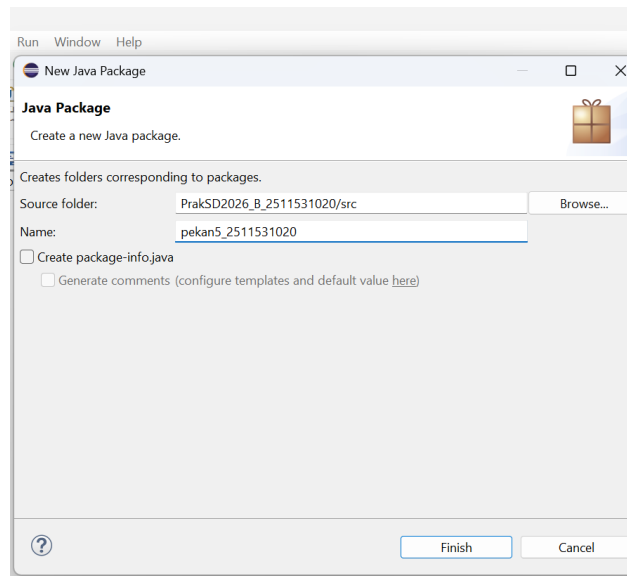
2.2.2 Membuat Java Project

1. Klik file
2. Klik java project



Gambar 2.2 Java Project

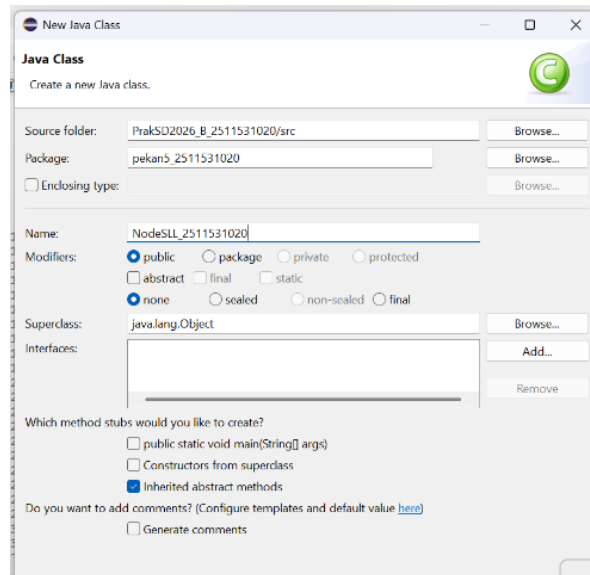
3. Buat nama project



Gambar 2.3 Nama Project

2.2.3 Class 1 “NodeSLL_2511531020”

1. Membuat package pekan5_2511531020 dan class NodeSLL_2511531020



Gambar 2.4 Membuat Nama package dan class NodeSLL_2511531020

2. Klik Finish
3. Kode Program

```
1 package pekan5_2511531020;
2
3 public class NodeSLL_2511531020 {
4     // node bagian data
5     int data_1020;
6     //pointer ke node berikutnya
7     NodeSLL_2511531020 next_1020;
8     //Konstruktur menginisialisasi node dengan data
9     public NodeSLL_2511531020 (int data_1020)
10    {
11        this.data_1020 = data_1020;
12        this.next_1020 = null;
13    }
14 }
15
16
```

Gambar 2.5 Class 1 “NodeSLL_2511531020”

4. Analisis Langkah
 - a. Deklarasi Variabel

- `front_1020` → Menunjuk indeks elemen paling depan (head) dari antrian.
- `rear_1020` → Menunjuk indeks elemen paling belakang (tail) dari antrian.
- `size_1020` → Menyimpan jumlah elemen yang sedang ada di antrian.
- `capacity_1020` → Menentukan kapasitas maksimum antrian.
- `array_1020[]` → Array yang digunakan untuk menyimpan data antrian.

Bagian ini mendefinisikan struktur dasar queue.

b. Konstruktor

- Menginisialisasi kapasitas antrian sesuai parameter.
- `front_1020` dan `size_1020` dimulai dari 0.
- `rear_1020` di-set ke `capacity - 1` agar perhitungan indeks modular bisa berjalan.
- Membuat array baru dengan ukuran sesuai kapasitas.

Konstruktor memastikan queue siap digunakan sejak awal.

c. Cek Kondisi

- `isFull_1020()` → Mengembalikan true jika `size_1020 == capacity_1020`.
- `isEmpty_1020()` → Mengembalikan true jika `size_1020 == 0`.

Berguna untuk mencegah operasi ilegal (enqueue saat penuh, dequeue saat kosong).

d. Menambah Data (Enqueue)

- Mengecek apakah antrian penuh.
- Jika tidak penuh, geser `rear_1020` ke indeks berikutnya dengan operasi modular $(rear_1020 + 1) \% capacity_1020$.
- Masukkan data baru ke posisi `rear_1020`.
- Tambahkan `size_1020`.

Data masuk dari belakang sesuai prinsip FIFO.

e. Menghapus Data (Dequeue)

- Mengecek apakah antrian kosong.
- Jika tidak kosong, ambil elemen di `front_1020`.
- Geser `front_1020` ke indeks berikutnya ($\text{front_1020} + 1$) % `capacity_1020`.
- Kurangi `size_1020`.
- Kembalikan elemen yang dihapus.

Data keluar dari depan sesuai prinsip FIFO.

f. Melihat Elemen Depan & Belakang

- `front_1020()` → Mengembalikan elemen paling depan.
- `rear_1020()` → Mengembalikan elemen paling belakang.

Berguna untuk memantau isi antrian tanpa menghapus data.

g. Menampilkan Isi Antrian

- Mengecek apakah antrian kosong.
- Jika tidak kosong, mencetak isi antrian dari `front_1020` sampai `rear_1020`.

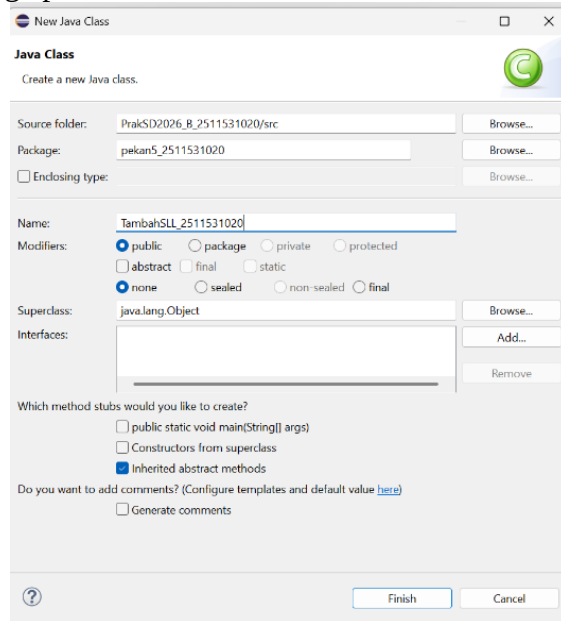
Memberikan visualisasi isi queue saat program berjalan.

h. Kesimpulan Analisis

- Kode sudah sesuai kategori tugas queue dengan array.
- Semua operasi dasar queue (`enqueue`, `dequeue`, cek kosong/penuh, lihat depan/belakang, `display`) sudah ada.
- Prinsip FIFO (First In, First Out) diterapkan dengan benar.
- Penamaan variabel dan class sudah sesuai aturan tugas (pakai NIM dan suffix `_1020`).

2.2.4 Class 2 “TambahSLL_2511531020”

1. Membuat package pekan5_2511531020 dan class TambahSLL_2511531020



Gambar 2.6 Membuat Nama package dan class TambahSLL_2511531020

2. Klik Finish

a. Kode Program

```
1 package pekan5_2511531020;
2
3 public class TambahSLL_2511531020 {
4     public static NodeSLL_2511531020 insertAtFront_1020(NodeSLL_2511531020 head_1020, int value_1020) {
5         NodeSLL_2511531020 newNode_1020 = new NodeSLL_2511531020(value_1020);
6         newNode_1020.next_1020 = head_1020;
7         return newNode_1020;
8     }
9
10    // fungsi menambahkan node di akhir SLL
11    public static NodeSLL_2511531020 insertAtEnd_1020(NodeSLL_2511531020 head_1020, int value_1020) {
12        // buat sebuah node dengan sebuah nilai
13        NodeSLL_2511531020 newNode_1020 = new NodeSLL_2511531020(value_1020);
14
15        // jika list kosong maka node jadi head
16        if (head_1020 == null) {
17            return newNode_1020;
18        }
19
20        // simpan head ke variabel sementara
21        NodeSLL_2511531020 last_1020 = head_1020;
22
23        // telusuri ke node akhir
24        while (last_1020.next_1020 != null) {
25            last_1020 = last_1020.next_1020;
26        }
27
28        // ubah pointer
29        last_1020.next_1020 = newNode_1020;
30        return head_1020;
31    }
32
33    // fungsi membuat node baru
34    static NodeSLL_2511531020 GetNode_1020(int data_1020) {
35        return new NodeSLL_2511531020(data_1020);
36    }
37 }
```

```

36     }
37 }
38 static NodeSLL_2511531020 insertPos_1020(NodeSLL_2511531020 headNode_1020, int position_1020, int value_1020) {
39     NodeSLL_2511531020 head_1020 = headNode_1020;
40     if (position_1020 < 1)
41         System.out.print("Invalid position");
42     if (position_1020 == 1) {
43         NodeSLL_2511531020 new_node_1020 = new NodeSLL_2511531020(value_1020);
44         new_node_1020.next_1020 = head_1020;
45         return new_node_1020;
46     } else {
47         while (position_1020-- != 0) {
48             if (position_1020 == 1) {
49                 NodeSLL_2511531020 newNode_1020 = getNode_1020(value_1020);
50                 new_node_1020.next_1020 = headNode_1020.next_1020;
51                 headNode_1020.next_1020 = newNode_1020;
52                 break;
53             }
54             headNode_1020 = headNode_1020.next_1020;
55         }
56         if (position_1020 != 1)
57             System.out.print("Posisi di luar jangkauan");
58         return head_1020;
59     }
60 }
61 }
62 public static void printList_1020(NodeSLL_2511531020 head_1020) {
63     NodeSLL_2511531020 curr_1020 = head_1020;
64     while (curr_1020.next_1020 != null) {
65         System.out.print(curr_1020.data_1020 + "-->");
66         curr_1020 = curr_1020.next_1020;
67     }
68     if (curr_1020.next_1020 == null) {
69         System.out.print(curr_1020.data_1020);
70     }
71     System.out.println();
72 }
73 public static void main (String [] args) {
74     // buat linked list 2->3->4->5->6
75     NodeSLL_2511531020 head_1020 = new NodeSLL_2511531020(2);
76     head_1020.next_1020 = new NodeSLL_2511531020(3);
77     head_1020.next_1020.next_1020 = new NodeSLL_2511531020(5);
78     head_1020.next_1020.next_1020.next_1020 = new NodeSLL_2511531020(6);
79     // cetak list asli
80     System.out.print("Senarai berantai awal:");
81     printList_1020(head_1020);
82     // tambahkan node baru di depan
83     System.out.print("tambah 1 simpul di depan: ");
84     int data_1020 = 1;
85     head_1020 = insertAtFront_1020(head_1020, data_1020);
86
87     // cetak update list
88     printList_1020(head_1020);
89
90     // tambahkan node baru di belakang
91     System.out.print("tambah 1 simpul di belakang: ");
92     int data2_1020 = 7;
93     head_1020 = insertAtEnd_1020(head_1020, data2_1020);
94
95     // cetak update list
96     printList_1020(head_1020);
97
98     System.out.print("tambah 1 simpul ke data 4: ");
99     int data3_1020 = 4;
100    int pos_1020 = 4;
101    head_1020 = insertPos_1020(head_1020, pos_1020, data3_1020);
102
103    // cetak update list
104    printList_1020(head_1020);
105 }

```

Gambar 2.7 Class 2 “TambahSLL_2511531020”

3. Analisis Langkah

a. Deklarasi Class

java

public class TambahSLL_2511531020 { ... }

- Class ini berisi kumpulan fungsi untuk operasi dasar pada Single Linked List (SLL).

b. Insert di Depan (insertAtFront_1020)

- Membuat node baru dengan nilai value_1020.

- Node baru menunjuk ke head_1020 lama.
 - Node baru menjadi head baru. Digunakan untuk menambah elemen di awal list.
- c. Insert di Belakang (insertAtEnd_1020)
- Membuat node baru dengan nilai value_1020.
 - Jika list kosong, node baru langsung jadi head.
 - Jika tidak kosong, telusuri sampai node terakhir (last_1020).
 - Hubungkan node terakhir ke node baru. Digunakan untuk menambah elemen di akhir list.
- d. Membuat Node Baru (getNode_1020)
- Fungsi sederhana untuk membuat node baru dengan data tertentu. Mempermudah pembuatan node tanpa harus menulis konstruktor secara langsung.
- e. Insert di Posisi Tertentu (insertPos_1020)
- Mengecek apakah posisi valid.
 - Jika posisi = 1, node baru ditambahkan di depan.
 - Jika posisi > 1, lakukan traversal sampai posisi yang ditentukan.
 - Sisipkan node baru di antara node yang ada. Digunakan untuk menambah elemen di posisi tertentu dalam list.
- f. Print List (printList_1020)
- Menelusuri list dari head hingga akhir.
 - Mencetak data setiap node dengan tanda panah -->.
 - Node terakhir dicetak tanpa panah. Memberikan visualisasi isi linked list.
- g. Main Method
- Membuat linked list awal: 2 -> 3 -> 5 -> 6.
 - Cetak list asli.
 - Tambahkan node di depan (1), hasil: 1 -> 2 -> 3 -> 5 -> 6.
 - Tambahkan node di belakang (7), hasil: 1 -> 2 -> 3 -> 5 -> 6 -> 7.

- Tambahkan node di posisi ke-4 dengan data 4, hasil: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7.

Main method menunjukkan uji coba semua operasi insert.

h. Kesimpulan Analisis

- Kode ini sudah mencakup operasi dasar insert pada SLL: depan, belakang, dan posisi tertentu.
- Fungsi printList_1020 membantu memverifikasi hasil setiap operasi.
- Prinsip dinamis linked list diterapkan dengan benar (node saling terhubung dengan pointer).
- Penamaan class, method, dan variabel sudah sesuai aturan tugas (pakai NIM + _1020).

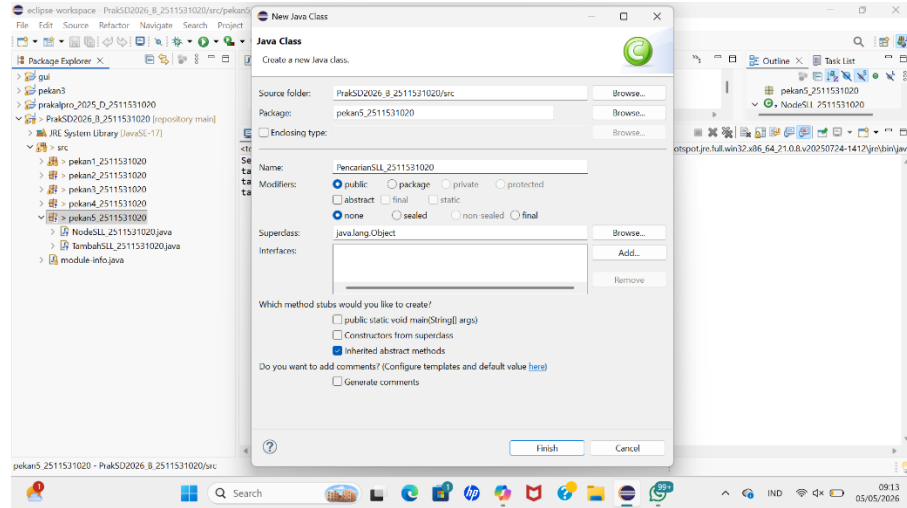
4. Output

```
<terminated> TambahSLL_2511531020 [Java Application] C:\Users\USER\p2
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

Gambar 2.8 Output

2.2.5 Class 3 “PencarianSLL_2511531020”

1. Membuat package pekan5_2511531020 dan class PencarianSLL_2511531020



Gambar 2.9 Membuat Nama package dan class 3
PencarianSLL_2511531020

2. Klik Finish
3. Kode Program

```
1 package pekan5_2511531020;
2
3 public class PencarianSLL_2511531020 {
4     static boolean searchKey_1020 (NodeSLL_2511531020 head_1020, int key) {
5         NodeSLL_2511531020 curr = head_1020;
6         while (curr != null) {
7             if (curr.data_1020 == key)
8                 return true;
9             curr = curr.next_1020; }
10        return false; }
11    public static void traversal_1020 (NodeSLL_2511531020 head_1020) {
12        // mulai dari head
13        NodeSLL_2511531020 curr = head_1020;
14        // telusuri sampai pointer null
15        while (curr != null) {
16            System.out.print(" " + curr.data_1020);
17            curr = curr.next_1020; }
18        System.out.println();
19    }
20
21    public static void main_1020(String[] args_1020) {
22        NodeSLL_2511531020 head_1020 = new NodeSLL_2511531020(14);
23        head_1020.next_1020 = new NodeSLL_2511531020(21);
24        head_1020.next_1020.next_1020 = new NodeSLL_2511531020(13);
25        head_1020.next_1020.next_1020.next_1020 = new NodeSLL_2511531020(30);
26        head_1020.next_1020.next_1020.next_1020.next_1020 = new NodeSLL_2511531020(10);
27
28        System.out.print("Penelusuran SLL : ");
29        traversal_1020(head_1020);
30
31        // data yang akan dicari
32        int key_1020 = 30;
33        System.out.print("cari data " + key_1020 + " = ");
34        if (searchKey_1020(head_1020, key_1020))
35            System.out.println("ketemu");
```

```

36         else
37             System.out.println("tidak ada");
38     }
39 }

```

Gambar 2.10 Class 3 “PencarianSLL_2511531020”

4. Analisis Langkah

a. Deklarasi Class

java

```
public class PencarianSLL_2511531020 { ... }
```

- Class ini berisi fungsi untuk pencarian data dan penelusuran (traversal) pada Single Linked List.

b. Fungsi Pencarian (searchKey_1020)

java

```
static boolean searchKey_1020(NodeSLL_2511531020 head_1020,
int key) { ... }
```

- Mulai dari node head.
- Telusuri node satu per satu menggunakan while (curr != null).
- Jika curr.data_1020 == key, kembalikan true.
- Jika sudah sampai akhir list (null) dan tidak ketemu, kembalikan false.

Fungsi ini mengimplementasikan linear search pada linked list.

c. Fungsi Traversal (traversal_1020)

java

```
public static void traversal_1020(NodeSLL_2511531020
head_1020) { ... }
```

- Mulai dari head.
- Telusuri semua node hingga null.
- Cetak setiap data node dengan spasi.
- Setelah selesai, cetak baris baru.

Fungsi ini digunakan untuk menampilkan isi linked list secara berurutan.

d. Main Method (main_1020)

java

```
public static void main_1020(String[] args_1020) { ... }
```

- Membuat linked list awal: 14 -> 21 -> 13 -> 30 -> 10.
- Cetak isi list dengan fungsi `traversal_1020`.
- Tentukan data yang akan dicari (`key_1020 = 30`).
- Panggil `searchKey_1020` untuk mencari data tersebut.
- Jika ketemu, tampilkan "ketemu". Jika tidak, tampilkan "tidak ada".

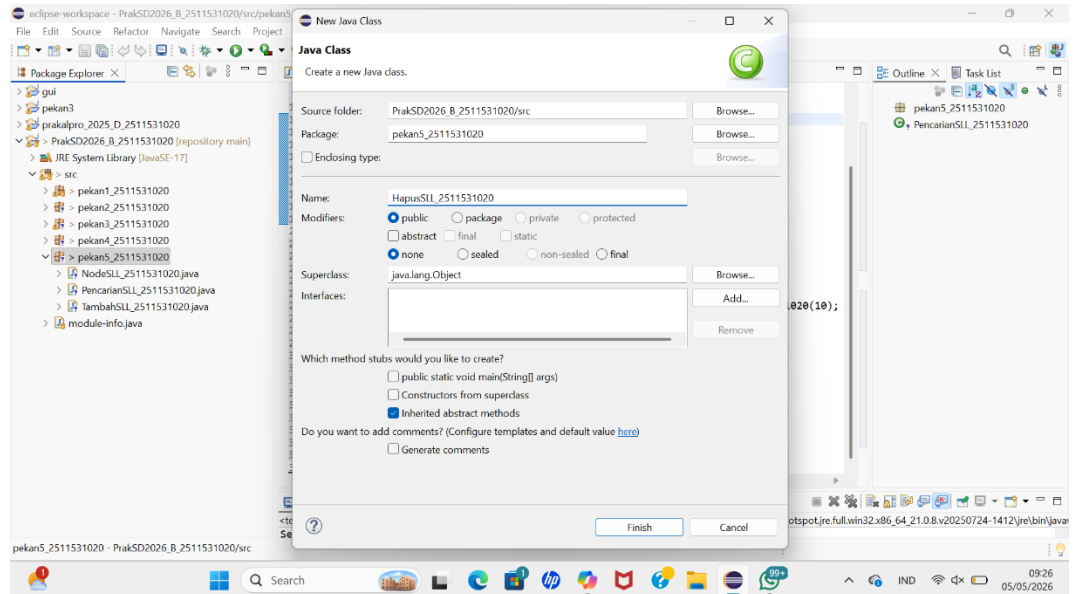
Main method ini menguji fungsi traversal dan pencarian.

e. Kesimpulan Analisis

- Kode ini sudah mencakup operasi traversal dan pencarian data pada SLL.
- Pencarian dilakukan dengan metode linear search karena linked list tidak memiliki indeks seperti array.
- Traversal memastikan semua node bisa ditampilkan dari head hingga tail.
- Penamaan class, method, dan variabel sudah sesuai aturan tugas (pakai NIM + _1020).
- Prinsip dasar linked list diterapkan dengan benar: data disimpan dalam node, dihubungkan dengan pointer `next_1020`, dan ditelusuri satu per satu.

2.2.6 Class 4 “HapusSLL_2511531020”

1. Membuat package pekan5_2511531020 dan class HapusSLL_2511531020



Gambar 2.11 Membuat Nama package dan class HapusSLL_2511531020

2. Klik Finish
3. Kode Program

```
1 package pekan5_2511531020;
2
3 public class HapusSLL_2511531020 {
4     // fungsi untuk menghapus head
5     public static NodeSLL_2511531020 deleteHead_1020(NodeSLL_2511531020 head_1020) {
6         // jika SLL kosong
7         if (head_1020 == null) {
8             return null;
9         }
10        // pindahkan head ke node berikutnya
11        head_1020 = head_1020.next_1020;
12        // return head baru
13        return head_1020;
14    }
15
16    // fungsi menghapus node terakhir SLL
17    public static NodeSLL_2511531020 removeLastNode_1020(NodeSLL_2511531020 head_1020) {
18        // jika list kosong , return null
19        if (head_1020 == null) {
20            return null;
21        }
22        // jika list satu node , hapus node dan return null
23        if (head_1020.next_1020 == null) {
24            return null;
25        }
26        // temukan node terakhir kedua
27        NodeSLL_2511531020 secondLast = head_1020;
28        while (secondLast.next_1020.next_1020 != null) {
29            secondLast = secondLast.next_1020;
30        }
31        // hapus node terakhir
32        secondLast.next_1020 = null;
33        return head_1020;
34    }
35 }
```

```

36 // fungsi menghapus node di posisi tertentu
37 public static NodeSLL_2511531020 deleteNode_1020(NodeSLL_2511531020 head_1020, int position_1020) {
38     NodeSLL_2511531020 temp_1020 = head_1020;
39     NodeSLL_2511531020 prev_1020 = null;
40
41     // jika linked list null
42     if (temp_1020 == null)
43         return head_1020;
44
45     // kasus 1: head dihapus
46     if (position_1020 == 1) {
47         head_1020 = temp_1020.next_1020;
48         return head_1020;
49     }
50
51     // kasus 2: menghapus node di tengah
52     for (int i_1020 = 1; temp_1020 != null && i_1020 < position_1020; i_1020++) {
53         prev_1020 = temp_1020;
54         temp_1020 = temp_1020.next_1020;
55     }
56
57     // jika ditemukan, hapus node
58     if (temp_1020 != null) {
59         prev_1020.next_1020 = temp_1020.next_1020;
60     } else {
61         System.out.println("Data tidak ada");
62     }
63     return head_1020;
64 }
65
66 // fungsi mencetak SLL
67 public static void printList_1020(NodeSLL_2511531020 head_1020) {
68     NodeSLL_2511531020 curr_1020 = head_1020;
69     while (curr_1020.next_1020 != null) {
70         System.out.print(curr_1020.data_1020 + "-->");
71         curr_1020 = curr_1020.next_1020;
72     }
73     System.out.print(curr_1020.data_1020);
74     System.out.println();
75 }
76
77 // kelas main
78 public static void main(String[] args) {
79     // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
80     NodeSLL_2511531020 head_1020 = new NodeSLL_2511531020(1);
81     head_1020.next_1020 = new NodeSLL_2511531020(2);
82     head_1020.next_1020.next_1020 = new NodeSLL_2511531020(3);
83     head_1020.next_1020.next_1020.next_1020 = new NodeSLL_2511531020(4);
84     head_1020.next_1020.next_1020.next_1020.next_1020 = new NodeSLL_2511531020(5);
85     head_1020.next_1020.next_1020.next_1020.next_1020.next_1020 = new NodeSLL_2511531020(6);
86
87     // cetak list awal
88     System.out.println("List awal: ");
89     printList_1020(head_1020);
90
91     // hapus head
92     head_1020 = deleteHead_1020(head_1020);
93     System.out.println("List setelah head dihapus: ");
94     printList_1020(head_1020);
95
96     // hapus node terakhir
97     head_1020 = removeLastNode_1020(head_1020);
98     System.out.println("List setelah simpul terakhir dihapus: ");
99     printList_1020(head_1020);
100
101     // Deleting node at position 2
102     int position_1020 = 2;
103     head_1020 = deleteNode_1020(head_1020, position_1020);
104     //print list after deletion
105     System.out.println("List setelah posisi 2 dihapus: ");
106     printList_1020(head_1020);
107 }
108 }
109

```

Gambar 2.12 Class 4 “HapusSLL_2511531020”

4. Analisis Langkah

a. Fungsi deleteHead_1020

- Mengecek apakah list kosong (head_1020 == null). Jika kosong, return null.

- Jika tidak kosong, head dipindahkan ke node berikutnya (head_1020.next_1020).
 - Mengembalikan head baru. Digunakan untuk menghapus node pertama (head).
- b. Fungsi removeLastNode_1020
- Mengecek apakah list kosong → return null.
 - Jika list hanya berisi satu node → hapus node dan return null.
 - Jika lebih dari satu node, telusuri hingga node kedua terakhir (secondLast).
 - Putuskan pointer ke node terakhir (secondLast.next_1020 = null).
 - Return head. Digunakan untuk menghapus node terakhir (tail).
- c. Fungsi deleteNode_1020
- Mengecek apakah list kosong.
 - Jika posisi = 1 → hapus head dengan memindahkan ke node berikutnya.
 - Jika posisi > 1 → lakukan traversal sampai posisi yang diinginkan.
 - Simpan node sebelumnya (prev_1020) dan node yang akan dihapus (temp_1020).
 - Hubungkan prev_1020.next_1020 ke temp_1020.next_1020 sehingga node target terlepas.
 - Jika posisi tidak valid → tampilkan pesan "Data tidak ada". Digunakan untuk menghapus node di posisi tertentu.
- d. Fungsi printList_1020
- Menelusuri list dari head hingga akhir.
 - Cetak data setiap node dengan tanda panah -->.
 - Node terakhir dicetak tanpa panah. Memberikan visualisasi isi linked list.
- e. Main Method
- Membuat linked list awal: 1 -> 2 -> 3 -> 4 -> 5 -> 6.
 - Cetak list awal.

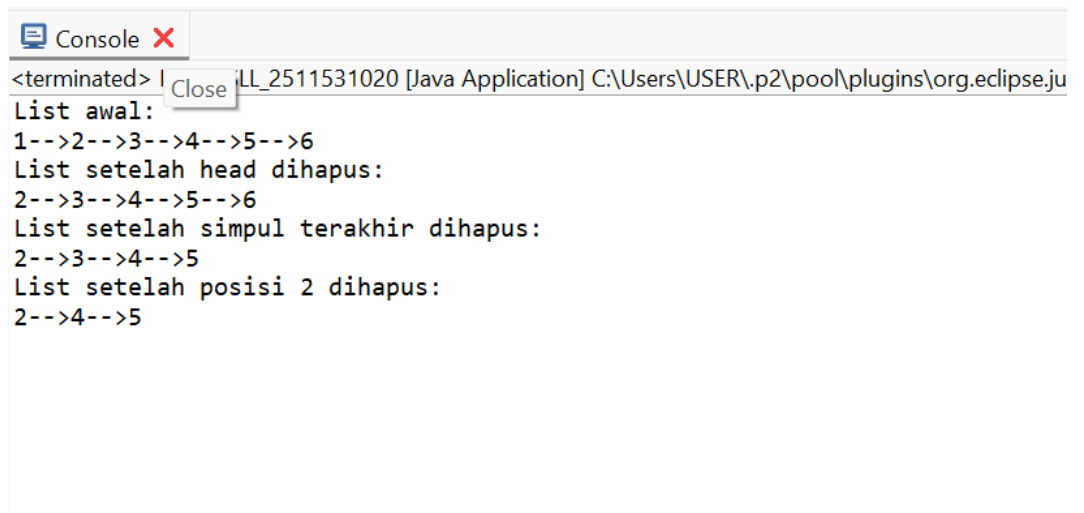
- Hapus head → hasil: 2 -> 3 -> 4 -> 5 -> 6.
- Hapus node terakhir → hasil: 2 -> 3 -> 4 -> 5.
- Hapus node di posisi ke-2 → hasil: 2 -> 4 -> 5.

Main method ini menguji semua operasi penghapusan: head, tail, dan posisi tertentu.

f. Kesimpulan Analisis

- Kode ini sudah mencakup operasi dasar delete pada SLL: head, tail, dan posisi tertentu.
- Fungsi printList_1020 membantu memverifikasi hasil setiap operasi.
- Prinsip linked list diterapkan dengan benar: node dihapus dengan cara memutus pointer, bukan menggeser elemen seperti array.
- Penamaan class, method, dan variabel sudah sesuai aturan tugas (pakai NIM + _1020).

5. Output



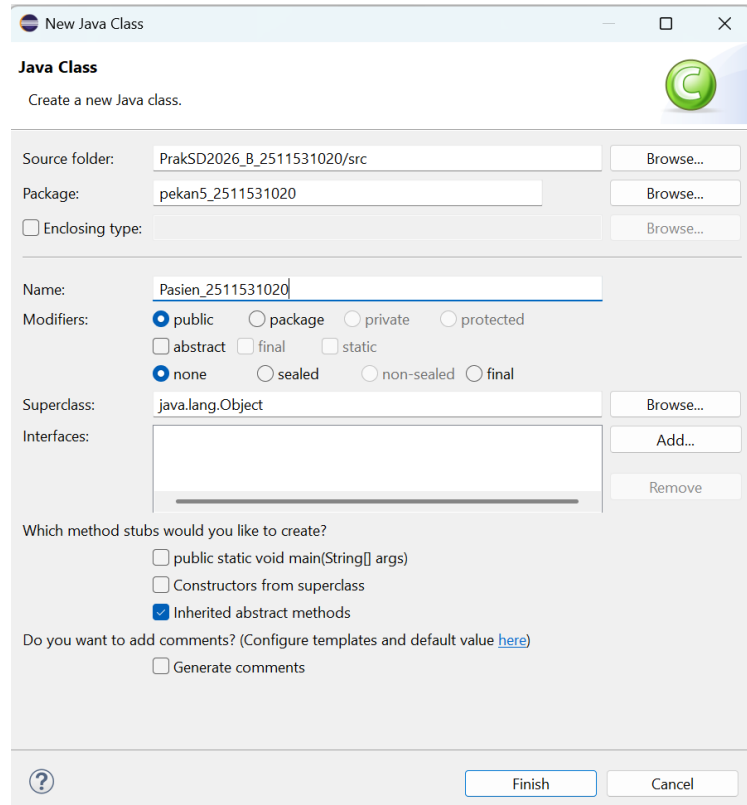
```
<terminated> I... LL_2511531020 [Java Application] C:\Users\USER\.p2\pool\plugins\org.eclipse.ju
List awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

Gambar 2.13 Output

2.3 Tugas Praktikum

2.3.1 Class 1 “Pasien_2511531020”

1. Membuat Nama package pekan4_2511531020 dan class Pasien_2511531020



Gambar 2.14 Membuat Nama package dan class Pasien_2511531020

2. Klik Finish

3. Kode Program

```
1 package pekan5_2511531020;
2
3 public class Pasien_2511531020 {
4     // Atribut
5     private String namaPasien_1020;
6     private String penyakit_1020;
7     private int nomorAntrian_1020;
8     Pasien_2511531020 next_1020; // pointer ke pasien berikutnya
9
10    // Constructor
11    public Pasien_2511531020(String namaPasien_1020, String penyakit_1020, int nomorAntrian_1020) {
12        this.namaPasien_1020 = namaPasien_1020;
13        this.penakit_1020 = penyakit_1020;
14        this.nomorAntrian_1020 = nomorAntrian_1020;
15        this.next_1020 = null;
16    }
17
18    // Getter
19    public String getNamaPasien_1020() { return namaPasien_1020; }
20    public String getPenyakit_1020() { return penyakit_1020; }
21    public int getNomorAntrian_1020() { return nomorAntrian_1020; }
22    public Pasien_2511531020 getNext_1020() { return next_1020; }
23
24    // Setter
25    public void setNamaPasien_1020(String namaPasien_1020) { this.namaPasien_1020 = namaPasien_1020; }
26    public void setPenyakit_1020(String penyakit_1020) { this.penakit_1020 = penyakit_1020; }
27    public void setNomorAntrian_1020(int nomorAntrian_1020) { this.nomorAntrian_1020 = nomorAntrian_1020; }
28    public void setNext_1020(Pasien_2511531020 next_1020) { this.next_1020 = next_1020; }
29 }
30 }
```

Gambar 2.15 Class “Pasien_2511531020”

4. Analisis Langkah

a. Atribut:

- namaPasien_1020, penyakit_1020, nomorAntrian_1020
→ menyimpan data pasien.
- next_1020 → pointer ke node pasien berikutnya.

b. Constructor

- Menginisialisasi semua atribut saat objek pasien dibuat.
- next_1020 di-set null karena pasien baru belum terhubung ke node lain.

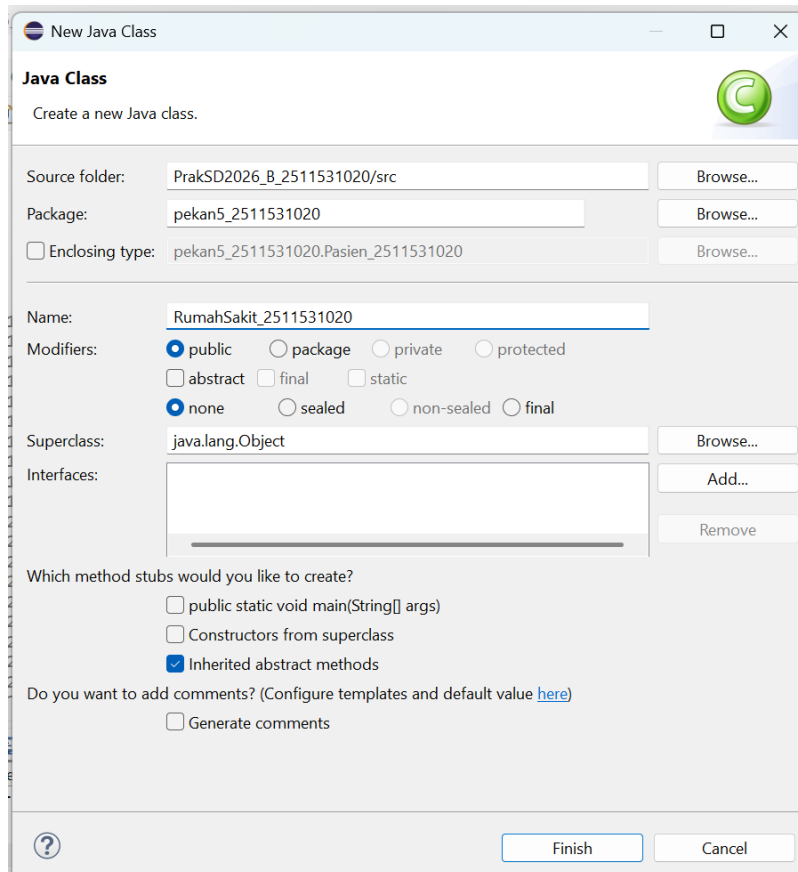
c. Getter & Setter

- Getter → mengambil nilai atribut.
- Setter → mengubah nilai atribut.
- Termasuk setter untuk next_1020 agar node bisa dihubungkan ke pasien berikutnya.

Kelas ini berfungsi sebagai ADT (Abstract Data Type) untuk merepresentasikan node pasien dalam linked list.

2.3.2 Class 2 “RumahSakit_2511531020”

1. Membuat Nama package pekan5_2511531020 dan class RumahSakit_2511531020



Gambar 2.16 Membuat Nama package dan class
RumahSakit_2511531020

2. Klik Finish
3. Kode Program

```

1 package pekan5_2511531020;
2
3 import java.util.Scanner;
4
5 public class RumahSakit_2511531020 {
6     private Pasien_2511531020 head_1020;
7     private int counter_1020; // nomor antrian auto-increment
8
9     public RumahSakit_2511531020() {
10         head_1020 = null;
11         counter_1020 = 0;
12     }
13
14     // Daftarkan Pasien (Insert at Tail)
15     public void daftarkanPasien_1020(String nama_1020, String penyakit_1020) {
16         counter_1020++;
17         Pasien_2511531020 newPasien_1020 = new Pasien_2511531020(nama_1020, penyakit_1020, counter_1020);
18         if (head_1020 == null) {
19             head_1020 = newPasien_1020;
20         } else {
21             Pasien_2511531020 temp_1020 = head_1020;
22             while (temp_1020.getNext_1020() != null) {
23                 temp_1020 = temp_1020.getNext_1020();
24             }
25             temp_1020.setNext_1020(newPasien_1020);
26         }
27         System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counter_1020);
28     }
29
30     // Panggil Pasien (Delete Head)
31     public void panggilPasien_1020() {
32         if (head_1020 == null) {
33             System.out.println("Antrian kosong!");
34             return;
35         }
36         System.out.println("Memanggil Pasien: " + head_1020.getNamaPasien_1020() +
37             " | Keluhan: " + head_1020.getPenyakit_1020() +
38             " | Nomor Antrian: " + head_1020.getNomorAntrian_1020());
39         head_1020 = head_1020.getNext_1020();
40     }
41
42     // Tampilkan Antrian (Display)
43     public void tampilkanAntrian_1020() {
44         if (head_1020 == null) {
45             System.out.println("Antrian kosong!");
46             return;
47         }
48         Pasien_2511531020 temp_1020 = head_1020;
49         System.out.println("=== Daftar Antrian Pasien ===");
50         while (temp_1020 != null) {
51             System.out.println("Nomor: " + temp_1020.getNomorAntrian_1020() +
52                 " | Nama: " + temp_1020.getNamaPasien_1020() +
53                 " | Keluhan: " + temp_1020.getPenyakit_1020());
54             temp_1020 = temp_1020.getNext_1020();
55         }
56     }
57
58     // Cari Pasien (Search by Name)
59     public void cariPasien_1020(String nama_1020) {
60         Pasien_2511531020 temp_1020 = head_1020;
61         while (temp_1020 != null) {
62             if (temp_1020.getNamaPasien_1020().equalsIgnoreCase(nama_1020)) {
63                 System.out.println("Pasien ditemukan! Nomor Antrian: " + temp_1020.getNomorAntrian_1020() +
64                     " | Nama: " + temp_1020.getNamaPasien_1020() +
65                     " | Keluhan: " + temp_1020.getPenyakit_1020());
66                 return;
67             }
68             temp_1020 = temp_1020.getNext_1020();
69         }
70         System.out.println("Pasien dengan nama " + nama_1020 + " tidak ditemukan.");
71     }
72
73     // Cek Status Antrian
74     public void cekStatusAntrian_1020() {
75         if (head_1020 == null) {
76             System.out.println("Antrian kosong!");
77             return;
78         }
79         int jumlah_1020 = 0;
80         Pasien_2511531020 temp_1020 = head_1020;
81         while (temp_1020 != null) {
82             jumlah_1020++;
83             temp_1020 = temp_1020.getNext_1020();
84         }
85         System.out.println("Jumlah pasien: " + jumlah_1020);
86         System.out.println("Pasien terdepan: " + head_1020.getNamaPasien_1020() +
87             " | Keluhan: " + head_1020.getPenyakit_1020());
88     }
89
90     // Main Program
91     public static void main(String[] args) {
92         Scanner sc_1020 = new Scanner(System.in);
93         RumahSakit_2511531020 rs_1020 = new RumahSakit_2511531020();
94         int pilihan_1020;
95
96         do {
97             System.out.println("\n=== Antrian Rumah Sakit NIM: 2511531020 ===");
98             System.out.println("1. Daftarkan Pasien");
99             System.out.println("2. Panggil Pasien");
100             System.out.println("3. Tampilkan Antrian");
101             System.out.println("4. Cari Pasien");
102             System.out.println("5. Cek Status Antrian");
103             System.out.println("6. Keluar");
104             System.out.print("Pilihan: ");
105             pilihan_1020 = sc_1020.nextInt();

```



```

106         sc_1020.nextLine(); // clear buffer
107
108     switch (pilihan_1020) {
109     case 1:
110         System.out.print("Masukkan Nama Pasien: ");
111         String nama_1020 = sc_1020.nextLine();
112         System.out.print("Masukkan Keluhan: ");
113         String penyakit_1020 = sc_1020.nextLine();
114         rs_1020.daftarkanPasien_1020(nama_1020, penyakit_1020);
115         break;
116     case 2:
117         rs_1020.panggilPasien_1020();
118         break;
119     case 3:
120         rs_1020.tampilkanAntrian_1020();
121         break;
122     case 4:
123         System.out.print("Masukkan Nama Pasien yang dicari: ");
124         String cari_1020 = sc_1020.nextLine();
125         rs_1020.cariPasien_1020(cari_1020);
126         break;
127     case 5:
128         rs_1020.cekStatusAntrian_1020();
129         break;
130     case 6:
131         System.out.println("Program selesai.");
132         break;
133     default:
134         System.out.println("Pilihan tidak valid!");
135     }
136     while (pilihan_1020 != 6);
137     sc_1020.close();
138 }
139 }
140

```

Gambar 2.17 Class “RumahSakit_2511531020”

4. Analisis Langkah

a) Deklarasi Atribut

- head_1020 → menunjuk ke pasien pertama dalam antrian.
- counter_1020 → menyimpan nomor antrian terakhir (auto-increment).

b) Daftarkan Pasien (Insert at Tail)

- Membuat node pasien baru dengan nomor antrian otomatis.
- Jika list kosong → pasien baru jadi head.
- Jika tidak kosong → traversal ke node terakhir, lalu hubungkan ke pasien baru.

c) Panggil Pasien (Delete Head)

- Mengecek apakah list kosong.
- Jika tidak kosong → tampilkan data pasien head, lalu geser head ke node berikutnya.

d) Tampilkan Antrian (Display)

- Traversal dari head hingga null.
- Cetak semua data pasien (nomor, nama, keluhan).

e) Cari Pasien (Search)

- Traversal dari head.
- Bandingkan nama pasien dengan input (case-insensitive).
- Jika ketemu → tampilkan data pasien. Jika tidak → tampilkan pesan “tidak ditemukan”.

f) Cek Status Antrian

- Hitung jumlah pasien dengan traversal.
- Tampilkan jumlah pasien dan data pasien terdepan.

g) Main Method

- Menyediakan menu interaktif: daftar pasien, panggil pasien, tampilkan antrian, cari pasien, cek status, keluar.
- Menggunakan Scanner untuk input dari user.

Kelas ini berfungsi sebagai driver yang mengelola operasi queue berbasis Single Linked List sesuai tema simulasi antrian rumah sakit.

5.Output

```
Console X
<terminated> RumahSakit_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotsp
Masukkan Keluhan: Batuk
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511531020 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 3
=== Daftar Antrian Pasien ===
Nomor: 1 | Nama: Budi Santoso | Keluhan: Demam
Nomor: 2 | Nama: Mutia Rani | Keluhan: Batuk

=== Antrian Rumah Sakit NIM: 2511531020 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Memanggil Pasien: Budi Santoso | Keluhan: Demam | Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511531020 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 5
Jumlah pasien: 1

Pasien terdepan: Mutia Rani | Keluhan: Batuk

=== Antrian Rumah Sakit NIM: 2511531020 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan Nama Pasien yang dicari: Mutia Rani
Pasien ditemukan! Nomor Antrian: 2 | Nama: Mutia Rani | Keluhan: Batuk

=== Antrian Rumah Sakit NIM: 2511531020 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan: 6
Program selesai.
```

Gambar 2.18 Output

BAB III

KESIMPULAN

Dimulai dari kelas dasar NodeSLL_2511531020 yang merepresentasikan node dengan atribut data dan pointer next_1020, kemudian dikembangkan menjadi berbagai operasi penting: TambahSLL untuk menambahkan node di depan, belakang, atau posisi tertentu; PencarianSLL untuk melakukan traversal dan pencarian data; serta HapusSLL untuk menghapus node di head, tail, maupun posisi tertentu. Semua operasi ini menunjukkan fleksibilitas linked list dalam mengelola data secara dinamis. Selanjutnya, konsep tersebut diaplikasikan dalam simulasi nyata melalui kelas Pasien_2511531020 dan RumahSakit_2511531020, yang mensimulasikan sistem antrian pasien dengan prinsip FIFO (First In, First Out). Program ini tidak hanya memenuhi aturan penamaan sesuai NIM, tetapi juga mencakup seluruh operasi yang diminta: insert, delete, display, search, dan cek status, dengan penanganan kondisi list kosong agar tidak error. Dengan demikian, keseluruhan implementasi dari NodeSLL hingga simulasi antrian rumah sakit berhasil menunjukkan pemahaman konsep linked list, penerapan operasi dasar, serta integrasi ke dalam kasus nyata yang relevan.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [2] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA: Addison-Wesley, 2011.
- [3] Oracle, “The Java™ Tutorials,” Oracle, 2024. [Online]. Available: <https://docs.oracle.com/javase/tutorial/>
- [4] E. Horowitz and S. Sahni, *Fundamentals of Data Structures*, New Delhi: Universities Press, 2008.